

A. Section-A

1. Is $P = NP$?

Ans Unknown. This is one of the most famous unsolved problem in computer science. It is widely believed that $P \neq NP$, but no formal proof exists yet.

2. What is decision problem?

Ans A decision problem is a computational problem that can be answered with a simple "yes" or "no" response. It involves determining whether a given input satisfies a specific condition or property.

3. Difference between P and NP problems.

Ans P problem: Problems that can be solved in polynomial time by a deterministic Turing machine.

NP problem - Problems that can be verified in polynomial time by a Non-deterministic Turing machine.

4. Explain the difference between NP-hard and NP-complete problems.

Ans NP-hard :- At least as hard as the hardest NP problems, may not be in NP.

NP-complete :- Both in NP and as hard as any NP problem.

5. How to check a problem in NP.

Ans To determine if a problem is in NP, we must verify two conditions:

i) The problem can be solved non-deterministically in polynomial time.

ii) Any proposed solution can be verified in

• Section-B ($3 \times 7 = 21$)

Q. State and prove Cook's Theorem.

Ans. Cook's Theorem states that the Boolean Satisfiability Problem (SAT) is NP-complete. This Theorem established the concept of NP-completeness and laid the foundation for the theory of computational complexity.

Statement:-

The Boolean satisfiability Problem (SAT) is NP-complete.

• Proof Outline:-

i) SAT is in NP

• Given a Boolean formula and a proposed solution (an assignment of variable), we can verify in polynomial time whether the formula evaluates to true.

• Thus, SAT is in NP.

ii) Reduction of any NP Problem to SAT.

To prove that SAT is NP-complete, we need to show that any problem in NP can be reduced to SAT in polynomial time.

For a problem $L \in NP$:

i) Non-deterministic Turing Machine (NDTM):

By definition:-

• Let M be the NDTM for L that runs in $P(n)$ time for input x .

2) Encoding the computation of NDTM as a Boolean Formula.

• Represent the computation of M as a sequence of configurations.

• Use Boolean variable to encode:-

→ Tape contents at each step.

→ State of the machine at each step.

→ Position of the tape head at each step.

• Construct Boolean formulae to ensure:-

→ valid initial config, Transition rules of M are followed, and final config.

3. Polynomial-Time Reduction -

The size of the boolean formula is polynomial in the size of the input x because the NDTM runs in polynomial time.

Hence, this reduction can be performed in polynomial time.

Thus, SAT is NP-complete. Proving Cook's algo.

7. Prove that CNF-Satisfiability is NP-complete.

Ans CNF-Satisfiability is in NP complete.

So

i) CNF-Satisfiability is in NP:-

- Given a CNF formula and a truth assignment, we can verify in polynomial time whether it satisfies the formula.
- Therefore, CNF-Satisfiability is in NP.

ii) Reduction from SAT to CNF-Satisfiability:-

- Any Boolean formula can be transformed into an equivalent formula in conjunctive Normal form (CNF) using polynomial time transformations like the Tseitin transformation.
- Since SAT is NP-complete (Cook's algo) and every instance of SAT can be reduced to CNF-Satisfiability in polynomial time, CNF-Satisfiability is at least as hard as SAT.

Conclusion:-

- CNF-Satisfiability is in NP.
- SAT reduces to CNF-Satisfiability in polynomial time.

Hence,

CNF-Satisfiability is NP-complete.

8- Prove that Set-Partition problem is NP-complete.

Ans Problem Definition

The set partition problem asks whether a given set of integers can be partitioned into two subsets such that the sum of elements in both subsets is equal.

• Proof outline:-

i) Show that the problem is in NP.

ii) Prove that it is NP-hard by reducing a known NP-complete problem to set-partition.

• Step 1:- Set-Partition is in NP.

• Given a partition of the set, we can compute the sums of the two subsets in polynomial time and verify if they are equal.

• Thus, the Set-Partition problem is in NP.

Step 2: Set-Partition is NP-hard

We reduce the Subset-Sum problem (which is NP-complete) to the Set-Partition problem.

(i) Subset-Sum Problem Definition:-

Given a set S of integers and a target T , determine if there exists a subset of S whose elements sum to T .

(ii) Reduction construction:-

• Given an instance of subset-sum (S, T) , construct an instance of set-partition S' by adding an extra element $x = 2T - \text{sum}(S)$.

• Define $S' = S \cup \{x\}$

(iii) Correctness:-

• If S has a subset S_1 that sums to T , the remaining elements of S plus x will sum to T , forming another partition of S' .

(iv) Polynomial Time:-

• The construction of S' and the verification of the reduction

are polynomial in the size of S .

Conclusion: -

- i) Set-Partition is in NP.
 - ii) Subset-Sum reduces to Set Partition in Polynomial time.
- Thus, the Set Partition Problem is NP-Complete.

• Section - C (3X11=33)

1. Is 3-CNF SAT Problem reducible to SAT Problem. Provide example.

Ans: Yes, 3-CNF SAT is reducible to SAT.

Reason:-

The 3-CNF SAT problem is a subset of the SAT Problem where each clause contains at most three literals. Since any 3-CNF formula is already a Boolean formula, it is naturally an instance of SAT. The reduction is straightforward and requires no transformation.

Example: 3-CNF SAT Formula:

Consider the 3-CNF SAT Formula:

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (x_2 \vee \neg x_3 \vee x_4)$$

• Equivalent SAT Formula:

The same formula is valid in SAT without any modification.

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (x_2 \vee \neg x_3 \vee x_4)$$

• Reduction process:

- i) input: 3-CNF SAT Formula Φ
- ii) output: Same Formula Φ as SAT Formula
- iii) so Φ preserved directly

• Proof of correctness

i) If 3-CNF SAT Formula is satisfiable, SAT Formula is satisfiable.

ii) If SAT formula is not satisfiable, 3-CNF SAT Formula is not satisfiable.

Q. Prove that travelling Salesman Problem is NP-complete.

Ans. Proof: Travelling Salesman Problem (TSP) is NP-complete

i) TSP is in NP -

- Given a solution (a specific order of cities), we can verify in polynomial time whether the total distance of the tour is less than or equal to a given value K .
- Thus TSP is in NP

ii) TSP is NP-hard (Reduction from Hamiltonian Cycle problem)

- The Hamiltonian Cycle Problem (HCP) is NP-complete. We reduce HCP to TSP to prove that TSP is NP-hard.

Hamiltonian Cycle Problem:

Given a graph, determine whether there is a cycle that visits each vertex exactly once.

• Reduction:

Given an instance of HCP, construct a TSP instance by setting all edge weights to 1. Set the target tour length to n , where $n = \text{no. of vertices}$.

- If there is a Hamiltonian cycle, the tour length will be n .
- If there is no Hamiltonian cycle, no tour of length n can exist.

→ This reduction from HCP to TSP can be done in polynomial time.

Conclusion:-

- TSP is in NP
- HCP reduces to TSP in Polynomial time

Thus,

The traveling Salesman Problem is NP-complete.

3. What is Vertex Cover Decision Problem. Prove that it is NP-complete.

The vertex cover decision problem asks:

Given a graph $G = (V, E)$ and an integer K is there a subset $V' \subseteq V$ such that $|V'| \leq K$ and every edge in E is incident to at least one vertex in V' ?

In other words,

the problem asks if we can select K or fewer vertices such that every edge in the graph is covered by at least one of the selected vertices.

• Proof: Vertex Cover is NP-complete

i) Vertex cover is in NP

• Given a set of vertices V' , we can check in polynomial time if:

1. $|V'| \leq K$ (the size of the set is at most K).

2. Every edge is covered.

Thus,

vertex cover is in NP.

ii) Vertex cover is NP-hard (Reduction from 3-SAT).

We reduce the 3-SAT problem (which is NP-complete) to the vertex cover problem to show that vertex cover is NP-hard.

• 3-SAT problem: -

Given a Boolean formula in 3-CNF form, determine if there exists an assignment of truth values to variables such that the formula is satisfied.

• Reduction:

Given an instance of 3-SAT, we construct a graph G where

1. variable: - for each variable x_i in the formula, create two vertices corresponding to the literal x_i and its negation $\neg x_i$.

2. Clauses: For each clause in the 3-CNF formula create a triangle (3 vertices connected) where each vertex corresponds to a literal in the clause. If the literal is x_i , connect it to the vertex corresponding to x_i . If the literal is $\neg x_i$ connect it to the vertex corresponding to $\neg x_i$.

3. Edges: Connect each literal vertex to the triangle of the corresponding clause.

• Target: Set $K = 3 \times \text{no. of clauses}$.

• Correctness:

- If the 3-SAT formula is satisfiable, there exists a truth assignment that can "cover" all the clauses.
- If there is a vertex cover of size K in the graph, this implies there is a truth assignment that satisfies the 3-SAT formula, as each clause will be covered by a literal vertex.

Thus, the 3-SAT problem can be reduced to the vertex cover problem in polynomial time.

So, vertex cover is in NP.

3-SAT reduces to vertex cover in polynomial time.

Therefore, the vertex cover decision problem is NP-complete.

□